Especificación Formal de un Sistema de Monitoreo de Temperatura en Almacenes de Productos Perecederos

Ccahuana Larota Joshep Antony
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
jccahuanala@unsa.edu.pe

Cuenca Gutierrez Jose Luis
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
jcuenca@unsa.edu.pe

Cordova Sellerico James Seus
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
jcordovas@unsa.edu.pe

Huayhua Mayta Ivan Rodrigo
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
ihuayhuam@unsa.edu.pe

Véliz Saihua Rodrigo Alejandro
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional de San Agustín
Arequipa, Perú
rvelizs@unsa.edu.pe

***Resumen*– El presente artículo aborda el diseño y especificación formal de un sistema de monitoreo de temperatura para almacenes de productos perecederos, empleando el método VDM++. Este sistema propone una solución escalable y automatizada que utiliza sensores para medir la temperatura en tiempo real, garantizando el cumplimiento de rangos óptimos de almacenamiento. Ante desviaciones de los límites preestablecidos, el sistema genera alertas que contribuyen a prevenir el deterioro de los productos y optimizar su conservación. La propuesta incluye el modelado de requisitos, un esquema de control basado en clases y una validación exhaustiva mediante herramientas como VDM++ Toolbox, asegurando la consistencia del modelo y su eficacia en entornos reales.**

Palabras clave-- monitoreo de temperatura, productos perecederos, especificación formal, VDM++, sistemas de almacenamiento.

I. Introducción

En el siempre exigente mundo de la industria alimentaria, el control de la temperatura en los almacenes se erige como una de las fronteras más delicadas para preservar la calidad y seguridad de los productos perecederos [1]. Un simple desvío en los valores térmicos puede acelerar, de manera inexorable, la degradación de los productos, afectando no solo su vida útil, sino también su valor comercial. Viene a la mente una imagen tan clara como preocupante: un tercio de los alimentos producidos a nivel mundial se pierde o desperdicia, según recientes estudios [2], tiene sus raíces en la negligencia o en la insuficiencia de los sistemas de almacenamiento, especialmente en lo que a temperatura se refiere. Un problema que se traduce, al final, en grandes pérdidas económicas para las empresas, y lo que es aún más inquietante, en un riesgo latente para la salud pública.

Parece insólito que, en una era de automatización creciente, muchos almacenes sigan dependiendo de métodos manuales o sistemas no integrados para controlar la temperatura [3]. Estas soluciones anacrónicas, tan propensas al error humano, rara vez permiten reaccionar a tiempo ante anomalías, dejando que los daños, como una sombra imperceptible, se instalen antes de ser detectados. La falta de escalabilidad en estos sistemas también impide su adaptabilidad a diferentes tipos de almacenes y productos, un desafío que limita la eficacia de todo el proceso logístico [4].

Ahí radica la necesidad de un cambio. Un sistema que no solo registre y monitorice en tiempo real las condiciones térmicas, sino que, como un guardián invisible, emita alertas tempranas ante cualquier desviación de los rangos predefinidos. Este proyecto propone el planteamiento de la implementación para un sistema de monitoreo de temperaturas en almacenes de productos perecederos que aproveche tecnologías avanzadas, sustentado en la rigurosidad formal de VDM++. La propuesta no solo es ambiciosa, sino necesaria: un diseño escalable que se adapte a diferentes configuraciones de almacenes, capaz de prever pérdidas y mitigar riesgos, este planteamiento busca, en cada almacén, preservar lo que tanto cuesta cosechar: la integridad de los productos que alimentan al mundo.

El resto del artículo está organizado de la siguiente manera: En la sección II se presentan algunos trabajos relacionados y definiciones. En la sección III se presentó el marco teórico. En la sección IV se describe la propuesta que define los requisitos, el diagrama de clases y el sistema de control para la detección de temperatura. En la sección V se describen las especificaciones formales desarrolladas en

VDM++. En la sección VI se analiza la validación del sistema desarrollado. Finalmente, en la sección VII se presenta la discusión del artículo, en la sección VIII se presentan las conclusiones y en la sección IX se presentan las recomendaciones basadas en las conclusiones de la investigación.

II. Antecedentes

A. Trabajos Relacionados

En esta sección se revisan varios trabajos relacionados al monitoreo de temperatura en almacenes de productos perecederos y la mejora de eficiencia en ese ámbito.

La tesis de Beltrán y Garzón [5], " Diseño de un sistema de alerta y monitoreo de instalaciones frigoríficas en tiempo real", presenta una solución para la automatización del control de temperatura en instalaciones frigoríficas, en el cual anteriormente dicha actividad se realizaba de forma manual.

También tenemos la investigación de Saha y Majumdar [6], propone un sistema IoT para monitorear la temperatura en centros de datos, utilizando el ESP8266 como microcontrolador y sensores que transmiten datos en tiempo real a un panel basado en la nube. Este sistema permite generar alertas por SMS y correo electrónico cuando las temperaturas superan los límites establecidos.

Por otro lado, el trabajo de Garrido, Jiménez, Toledo, Soto y Torres [7], "Trazador inteligente en tiempo real para productos perecederos", propone un trazador para monitorear condiciones ambientales clave en productos perecederos, tales como temperatura, humedad, concentración de gases, luminosidad y vibraciones. Este dispositivo combina múltiples sensores en una sola unidad, lo cual lo hace más rentable y eficiente que otras soluciones.

Además, en el trabajo de Wang et al. [8], se propone un sistema de monitoreo basado en sensores inalámbricos que utiliza la tecnología ZigBee para medir en tiempo real parámetros críticos como la temperatura, humedad y CO_2. Las pruebas demostraron una alta estabilidad en la comunicación, con una tasa de éxito en la transmisión de datos superior al 99%, optimizando así la gestión de productos perecederos.

Asimismo en la tesis de De Sousa y García [9] nos describe la búsqueda de cómo automatizar el monitoreo de la cadena de frío para empresas de alimentos. Usando sensores inalámbricos, el sistema registra datos clave, como temperaturas, para garantizar la calidad y conservación de los alimentos y finalmente se diseñó una red que facilita la integración y comunicación del sistema, mejorando los procesos de distribución y almacenamiento.

B. Definición de Producto Perecedero

De acuerdo con la FAO [10], los alimentos perecederos son aquellos expuestos a una rápida descomposición, deterioro o proliferación de microorganismos, ya sea con o sin producción de toxinas o metabolitos, cuando no se mantienen en las condiciones necesarias de almacenamiento. Este grupo incluye productos como leche, derivados lácteos, huevos, carne, aves de corral y pescado, entre otros. La conservación de estos productos requiere condiciones estrictas de almacenamiento, como temperaturas controladas, para prevenir riesgos a la salud pública y evitar pérdidas económicas [11].

C. Definición de Rangos de Temperatura Óptimos

Los rangos de temperatura óptimos son intervalos de temperatura en los que los productos alimenticios se mantienen seguros y conservan su calidad durante su almacenamiento. La refrigeración, generalmente entre 2 y 8 °C, ralentiza el crecimiento de microorganismos patógenos y preserva la frescura de muchos productos, mientras que el congelamiento a temperaturas de -18 °C o más bajas detiene casi por completo el crecimiento bacteriano, permitiendo que los alimentos se conserven por períodos más largos [12]. Sin embargo, el congelamiento solo mantiene la calidad de ciertos alimentos durante un tiempo limitado, ya que puede afectar su textura y sabor. Es crucial ajustar las condiciones de almacenamiento según el tipo de producto para garantizar su seguridad y calidad durante su vida útil [13].

III. Especificaciones Formales y VDM++

A. VDM++

El Método de Desarrollo de Viena (VDM) es un conjunto de notaciones y conceptos desarrollados en el Laboratorio de IBM en Viena. No se limita a una notación específica, sino que ofrece ideas aplicables a diversas notaciones. Originalmente, se utilizó para la descripción denotacional de lenguajes de programación y luego se extendió a la especificación de varios sistemas. [14]

VDM++ es una extensión orientada a objetos de VDM que incluye conceptos como clases, objetos, herencia y encapsulación, lo que lo hace ideal para modelar sistemas complejos. Se enfoca en la especificación orientada a modelos, utilizando tipos de datos simples y abstractos, así como invariantes para restringir pertenencias y especificaciones funcionales. Permite especificaciones tanto implícitas (pre/post) como explícitas (funcionales o imperativas). Incluye diversas técnicas de validación, como inspección, análisis estático, pruebas y verificación de modelos [15].

Es importante señalar que VDM no garantiza la "correctitud" absoluta de un programa, sino que permite verificar que cumple con una especificación formal, similar a cómo una prueba matemática demuestra un teorema [14].

B. Clases

Una clase VDM++ [16] consta de varias partes (Figura 1). Las variables de instancia, las operaciones y las funciones se pueden encontrar al principio de una clase. VDM++

también permite el uso de subprocesos que se comunican a través de objetos compartidos. El comportamiento de concurrencia se puede especificar en la sección de subprocesos y sincronización de una clase.

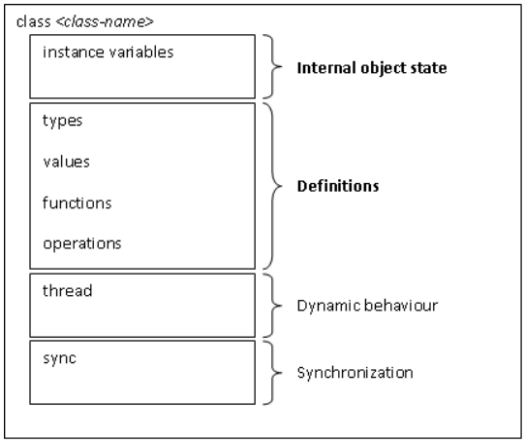


Figura 1. Estructura de Clase en VDM++, podemos ver que una clase en VDM++ tiene los siguientes bloques: a) Variables de Instancia, las cuales modelan el estado interno del objeto. b) Tipos: presenta la definición de los tipos de datos. c) Valores: permite la definición de constantes. d) Funciones: se define para describir una operación matemática o lógica que transforma entradas en salidas y f) Operaciones: define operaciones que pueden modificar las variables de instancia.

C. *Tipos*

En el modelo del sistema se emplean tipos básicos y compuestos. Entre los básicos se encuentran "real" para representar temperaturas y "seq of char" para cadenas de texto usadas en mensajes de alarma [17]. Los tipos compuestos incluyen conjuntos (set of), secuencias (seq of), y tipos enumerados como "<HighTemp>" y "<LowTemp>". Estos tipos son para estructurar el sistema y manipular datos como las temperaturas registradas por sensores, mensajes de alarma, y el conjunto de sensores instalados. Para la aplicación de nuestro modelo, queremos resaltar las operaciones predefinidas del tipo "seq of":

Operador	Semántica	Tipo
hd l	Primer elemento de la lista	seq of A -> A
tl l	Cola de la lista	seq of A -> seq of A
len l	Longitud de la lista	seq of A -> nat
inds l	Índices de la lista	seq of A -> set of nat
l(i)	Elemento en la posición i de la lista	seq of A * nat1 -> A
l1 = l2	Comparación de igualdad entre dos listas	seq of A * seq of A -> bool

Tabla 1. Tabla de operaciones predefinidas para el modelo.

D. *Expresiones*

Las expresiones en VDM++ son constructos utilizados para describir cálculos y evaluaciones dentro de un modelo formal, representando valores que pueden definir funciones, operaciones, condiciones y restricciones en el diseño de sistemas. Estas expresiones incluyen operaciones aritméticas, lógicas, de comparación y acceso a estructuras de datos como conjuntos, listas y mapas, además de expresiones más complejas como las cuantificadoras, que son esenciales para la definición de precondiciones, postcondiciones e invariantes. Existen dos tipos principales de expresiones cuantificadoras: el cuantificador universal (forall) y el existencial (exists), los cuales vinculan variables a un tipo de VDM++ o a valores pertenecientes a un conjunto, evaluándose frente a una expresión booleana. Cabe destacar que las expresiones en VDM++ no producen efectos secundarios, es decir, no afectan el valor de una variable de instancia, a menos que incluyan una llamada a una operación, y siempre se evalúan a un único valor, garantizando así la consistencia del modelo [18].

E. *Invariantes*

Las invariantes son restricciones que evitan que las variables de instancia de una clase tomen valores no permitidos. Estas representan condiciones que deben cumplirse siempre en el estado de un objeto. En VDM++, se utiliza la palabra reservada inv después de definir las variables de instancia [19].

F. *VDM++ Toolbox*

Es una herramienta diseñada para el análisis y desarrollo de modelos precisos de sistemas computacionales. Permite realizar análisis sintáctico, verificación de tipos, generación de condiciones de integridad y análisis de cobertura, además de ofrecer funcionalidades para validar especificaciones a través de un intérprete [20]. Incluye un entorno gráfico que permite depuración interactiva, ejecución de pruebas sistemáticas y generación automática de prototipos ejecutables. El analizador sintáctico comprueba la corrección de la sintaxis, reporta errores si los encuentra e intenta inferir posibles causas de estos errores. Cuando se garantiza tanto la corrección sintáctica como la de tipos, VDM++ToolBox permite validar la especificación ejecutando pruebas con un intérprete. Este intérprete ejecuta partes de la especificación usando valores definidos por el desarrollador como casos de prueba [19].

IV. Propuesta

El sistema evalúa las lecturas y determina si se han superado los límites permisibles de temperatura, en caso de exceder el límite, se activa la alarma para alertar sobre el problema. Este sistema contará con un sensor, que nos permitirá determinar en tiempo real la temperatura de los almacenes. En la figura 2 se muestra esta arquitectura.

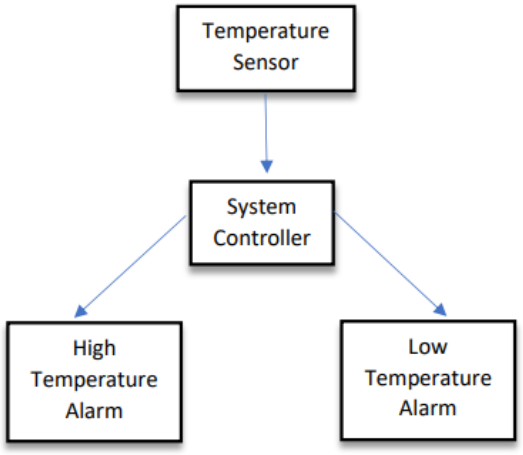


Figura 2. Arquitectura del sistema propuesto (elaborado por los autores), el sensor de temperatura informa si se han excedido los límites establecidos por producto y dispara las alarmas de acuerdo a si se ha excedido la temperatura máxima o si ha descendido por debajo de la temperatura mínima establecida.

A. Requerimientos

Para la determinación de los requerimientos en el sistema de monitoreo de temperatura en almacenes de productos perecederos, se tienen en cuenta los siguientes:

R1: El sistema debe recopilar datos de temperatura de los sensores: El sistema debe ser capaz de obtener y almacenar las lecturas de temperatura proporcionadas por los sensores para su posterior procesamiento y análisis.

R2: El sensor detectará temperaturas fuera de los rangos seguros: El sensor identificará si la temperatura medida se encuentra fuera de los límites seguros establecidos, ya sea por ser demasiado alta o demasiado baja.

R3: El sistema debe disparar alarmas automáticas cuando la temperatura supere los umbrales definidos: Cuando la temperatura detectada exceda los umbrales predefinidos (por ejemplo, -18°C o -23°C), el sistema activará automáticamente las alarmas correspondientes.

R4: El dispositivo debe contar con un alcance suficiente para medir temperaturas de manera efectiva: Los sensores deben ser capaces de medir temperaturas dentro de un rango amplio (por ejemplo, de -50°C a 50°C), cubriendo así los valores esperados en los ambientes de almacenamiento.

R5: La alarma permanecerá activa hasta que las temperaturas vuelvan a un rango seguro: Una vez activada por una temperatura fuera de rango, la alarma continuará activa hasta que las temperaturas se normalicen y entren dentro de los valores seguros, momento en el que la alerta será desactivada.

B. Diagrama de Clases

La figura 3 presenta el diagrama de clases propuesto, que es una representación básica del sistema en cuestión.

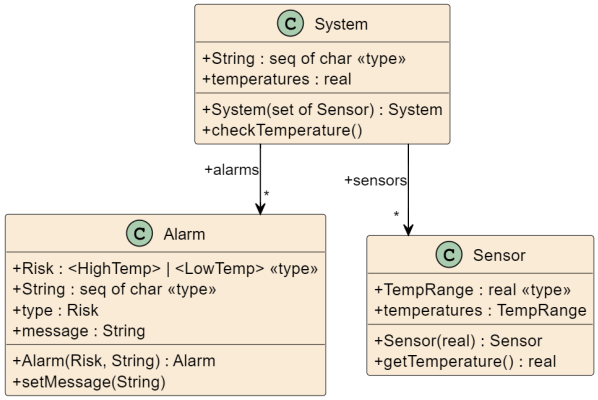


Figura 3. Diagrama de Clases propuesto del sistema de monitoreo compuesto por tres clases (elaborado por los autores). La principal es la clase Sistema, que funciona como intermediario en la comunicación entre la clase Sensor y Alarma, verificando constantemente que las temperaturas se encuentren dentro de los rangos establecidos. También gestiona el tipo de alarma ante una situación de riesgo.

C. Sistema de control

Primero, debemos identificar las áreas a monitorear, seleccionando aquellas donde exista un riesgo relacionado con la temperatura que pueda comprometer la conservación de productos perecederos. En este caso, utilizaremos un sensor de temperatura, como un termistor o un sensor RTD (Resistencia de Temperatura Dependiente), que mide exclusivamente la temperatura. Estos sensores son ideales para entornos donde las variaciones de temperatura son críticas para el almacenamiento de productos, y son capaces de proporcionar mediciones precisas y estables en un rango adecuado para ambientes fríos o calientes [21].

Cada sensor debe estar conectado al sistema, enviando mediciones de temperatura en tiempo real, normalmente expresadas en grados Celsius (°C). El sistema recibirá esta información constantemente y podrá identificar cuándo la temperatura alcanza niveles fuera del rango seguro para los productos almacenados.

El sistema debe incluir alarmas para advertir sobre condiciones de temperatura peligrosas, como temperaturas altas o bajas que puedan afectar la conservación de los productos. Estas alarmas, que se activan mediante altavoces, deben emitir un mensaje de advertencia cuando la temperatura superó los límites establecidos para el riesgo.

La lógica del sistema es que cada sensor de temperatura manejará una variable que almacenará el valor de la medición de temperatura. Esta variable cambiará de valor dependiendo de la fluctuación de la temperatura en el almacén. El sistema, al recibir estas mediciones, activará una alarma si la temperatura alcanza un nivel fuera del rango seguro, enviando un mensaje correspondiente al tipo de riesgo detectado.

V. Especificaciones Formales

En cuanto a la implementación en VDM++, la clase Sensor se modela en la Figura 4, para emular el comportamiento del sensor de temperatura que registra las mediciones. La Figura 5 muestra la clase Alarm, que modela el comportamiento de la alarma, la cual se activa cuando la temperatura está fuera del rango seguro. La clase principal es System, representada en la Figura 6, que modela el procesador central de todo el sistema y se encarga de gestionar las lecturas de los sensores y activar las alarmas cuando sea necesario.

```
1: class Sensor
2:     types
3:         public TempRange = real;
4:     instance variables
5:         public temperatures: TempRange;
6:     inv temperatures >= -50 and temperatures <= 50;
7:     operations
8:         public Sensor: real ==> Sensor
9:         Sensor(temp) == (
10:            temperatures := temp;
11:        );
12:        public getTemperature: () ==> real
13:        getTemperature() == (
14:            return temperatures;
15:        );
16: end Sensor
```

Figura 4. Clase Sensor en VDM++ (elaborado por los autores). En la línea 5, temperatures es la variable de instancia que almacenará la medición de la temperatura que el sensor detecta. Esta variable será de tipo TempRange, que se define como un valor real. La línea 3 define un tipo TempRange que no es nativo y almacena las mediciones de temperatura en grados Celsius, permitiendo así registrar los valores de temperatura en el sistema.

```
1: class Alarm
2:     types
3:         public Risk = <HighTemp> | <LowTemp>;
4:         public String = seq of char;
5:     instance variables
6:         public type: Risk;
7:         public message: String:= "none";
8:     inv message <> [];
9:     operations
10:        public Alarm: Risk * String ==> Alarm
11:        Alarm(risk, msg) == (
12:            type := risk;
13:            message := msg;
14:        );
15:        public setMessage: String ==> ()
16:        setMessage(msg) == (
17:            message := msg;
18:        );
19: end Alarm
```

Figura 5. Clase Alarma en VDM++ (elaborado por los autores). En las líneas 6 y 7 se observan las variables de instancia type, que define el tipo de riesgo gestionado por la alarma, y message, que contiene el mensaje que se reproducirá cuando exista peligro. Las líneas 3 y 4 definen los tipos Risk y String, que no son nativos. Risk puede ser de tipo HighTemp o LowTemp, mientras que String almacena el mensaje como una secuencia de caracteres.

```
1: class System
2:     types
3:         public String = seq of char;
4:     instance variables
5:         public sensors: set of Sensor;
6:         public alarms: seq of Alarm :=
7:         [new Alarm(<HighTemp>, "none"), new Alarm(<LowTemp>, "none")];
8:         public temperatures: real := 0;
9:     operations
10:        public System: set of Sensor ==> System
11:        System(sens) == (
12:            sensors := sens;
13:        );
14:        public checkTemperature: () ==> ()
15:        checkTemperature() == (
16:            for all s in set sensors do
17:                temperatures := s.getTemperature();
18:                if temperatures >= -18
19:                then alarms(1).setMessage("High temperature detected")
20:                else if temperatures <= -23
21:                then alarms(2).setMessage("Low temperature detected")
22:                else
23:                (
24:                    alarms(1).setMessage("none");
25:                    alarms(2).setMessage("none");
26:                )
27:        )
28: end System
```

Figura 6: Clase del sistema en VDM++. Las líneas 5 a 8 definen las variables de instancia para almacenar los sensores, las alarmas y las mediciones de temperatura. La variable de alarmas solo almacena dos tipos de alarmas: una para temperaturas altas y otra para temperaturas bajas. La línea 3 define el tipo de cadena de datos, que no es nativo. Las líneas 14 a 30 definen el método principal, el cual verifica si una determinada medición de temperatura está fuera del rango seguro, activando la alarma correspondiente en caso de riesgo.

VI. Validación

Se implementó la clase Test que se muestra en la Figura 7, donde se crearon los objetos necesarios de los tipos sensor, alarma y sistema, además de implementar los métodos correspondientes para su correcta creación y funcionamiento.

```
1: class Test
2: instance variables
3:     alarms: Alarm;
4:     sensors1: Sensor;
5:     sensors2: Sensor;
6:     sensors3: Sensor;
7:     system: System;
8: operations
9:     public run: () ==> System`String
10:    run() == test();
11:
12:    public test: () ==> System`String
13:    test() == (
14:        sensorTest();
15:
16:        -- Caso 1: Sensor con alta temperatura
17:        system := new System({sensors1});
18:        system.checkTemperature();
19:
20:        -- Caso 2: Sensor con baja temperatura
21:        system := new System({sensors2});
22:        system.checkTemperature();
23:
24:        -- Caso 3: Sensor con temperatura normal
25:        system := new System({sensors3});
26:        system.checkTemperature();
27:
28:        return "All tests passed";
29:    );
30:
31:    public sensorTest: () ==> ()
32:    sensorTest() == (
33:        sensors1 := new Sensor(-10); -- Alta temperatura
34:        sensors2 := new Sensor(-25); -- Baja temperatura
35:        sensors3 := new Sensor(-20); -- Temperatura normal
36:    );
37: end Test
```

Figura 7. Clase Test en VDM++ (elaborado por los autores). En la línea 12 se define la función test que llama a la función sensorTest, luego crea los objetos del sistema y llama a su método para verificar las temperaturas. Más adelante, en la línea 31 se define esta función sensorTest que se crean las distintas mediciones de temperatura.

La validación de las clases se realizó utilizando el intérprete VDM Toolbox. Los resultados de las pruebas se muestran en la Figura 8, donde se detallan las pruebas realizadas satisfactoriamente.

```
Initializing specification ... done
>> tcov reset
>> create test := new Test()
>> print test.run()
"All tests passed"
>> tcov write test.tc
>> rtinfo test.tc
 100%     1  Test`run
 100%     1  Test`test
 100%     1  Test`sensorTest
 100%  Test
 100%     6  Alarm`Alarm
 100%     4  Alarm`setMessage
 100%  Alarm
 100%     3  Sensor`Sensor
 100%     3  Sensor`getTemperature
 100%  Sensor
 100%     3  System`System
 100%     3  System`checkTemperature
 100%  System

Total Coverage: 100%
```

Figura 8. Resultados de las pruebas cubriendo el 100% de los casos, lo que valida y verifica el modelo planteado.

VII. Discusión

La inclusión de nuevas tecnologías de monitoreo y control automatizado podría representar una mejora sustancial en el almacenamiento de productos perecederos. La investigación demuestra que la implementación de sistemas avanzados de monitoreo de temperatura puede reducir significativamente las pérdidas económicas relacionadas con el deterioro de productos, y el uso de tecnologías inteligentes, como sensores, es una herramienta eficaz en el control de estos.

Aunque las tecnologías actuales son muy prometedoras, su integración en sistemas tradicionales de almacenamiento aún presenta retos logísticos y operativos. Se debe realizar el análisis de costos cuidadosamente para determinar la viabilidad de una transición total hacia sistemas automatizados, especialmente en empresas más pequeñas o en mercados donde el costo de implementación podría ser una limitante.

VIII. Conclusiones

Este trabajo proporcionó una base sólida para la mejora de los sistemas de monitoreo de temperatura en almacenes de productos perecederos, con el objetivo de reducir el riesgo de deterioro de los productos debido a temperaturas inadecuadas.

El sistema propuesto ha demostrado ser eficaz para garantizar la seguridad y calidad de los productos almacenados, minimizando los riesgos asociados con fluctuaciones de temperatura. El uso de especificaciones formales en el diseño y validación del software contribuye a asegurar la operación segura y eficiente del sistema, permitiendo una respuesta rápida y precisa ante cualquier anomalía.

La implementación de tecnologías de monitoreo de temperatura es necesario para mejorar la eficiencia en el almacenamiento de productos perecederos, y su aplicación puede resultar en una reducción significativa de las pérdidas económicas y riesgos para la salud pública.

IX. Recomendaciones

Se recomienda que las empresas que gestionan productos perecederos implementen y actualicen continuamente sistemas de monitoreo de temperatura, para garantizar un control preciso y minimizar el riesgo de deterioro de los productos debido a condiciones térmicas inadecuadas.

Se sugiere que al desarrollar e implementar sistemas de monitoreo, estas empresas utilicen especificaciones formales en el diseño y validación del software, y realicen pruebas de desempeño periódicas para verificar que los sistemas continúen funcionando correctamente ante posibles anomalías de temperatura.

Se debería invertir en la adopción de tecnologías de monitoreo y alerta temprana para optimizar la gestión de productos perecederos reduciendo pérdidas económicas, y permitiendo una intervención rápida en caso de fluctuaciones de temperatura, protegiendo tanto la calidad de los productos como la salud de los consumidores.

Referencias

[1] L. M. Paucar-Menacho, C. Moreno-Rojo y S. R. Chuqui-Diestra, “Emerging non-thermal technologies in the food industry: Advances and potential applications in food processing”, Sci. Agropecu., vol. 15, n.º 1, pp. 65–83, febrero de 2024. [En línea]. Disponible: https://doi.org/10.17268/sci.agropecu.2024.006

[2] Food and Agriculture Organization of the United Nations. “SOFA 2019 - El estado mundial de la agricultura y la alimentación”. www.fao.org. [En línea]. Disponible: https://www.fao.org/interactive/state-of-food-agriculture/2019/es/#:~:text=0%,cada%20año%20en%20el%20mundo

[3] A. Sander. “La automatización de almacenes y su impacto en la logística”. Blog sobre carretillas elevadoras, almacenaje y tecnología. Accedido el 22 de noviembre de 2024. [En línea]. Disponible: https://blog.toyota-forklifts.es/automatizacion-de-almacenes-impacto

[4] Esypro. “¿Por qué la escalabilidad es importante en un sistema logístico? - Esypro”. Esypro. Accedido el 22 de noviembre de 2024. [En línea]. Disponible: https://esypro.com/por-que-la-escalabilidad-es-importante-en-un-sistema-logistico/

[5] R. A Beltrán Villacís. & J. E.Garzón Ramírez . “Diseño de un sistema de alerta y monitoreo de instalaciones frigoríficas en tiempo real”. BS thesis. 2017. Disponible: http://www.dspace.espol.edu.ec/handle/123456789/42544

[6] S. Saha and A. Majumdar, “Data centre temperature monitoring with ESP8266 based Wireless Sensor Network and cloud based dashboard

with real time alert system," 2017 Devices for Integrated Circuit (DevIC), Mar. 2017, doi: https://doi.org/10.1109/devic.2017.8073958.
[7] J. Garrido López, M. Jiménez Buendía, A. Toledo Moreo, F. Soto Vallés y R. Torres Sánchez, "Trazador inteligente en tiempo real para productos perecederos", Jorn. Autom., n.º 45, julio de 2024. Disponible: https://doi.org/10.17979/ja-cea.2024.45.10860
[8] J. Wang et al., "Wireless sensor network for real-time perishable food supply chain management", Comput. Electron. Agriculture, vol. 110, pp. 196–207, enero de 2015. Disponible: https://doi.org/10.1016/j.compag.2014.11.009
[9] J. De Souza y C. Garcia, "Sistema de Monitoreo en Tiempo Real para el Control de Parámetros de la Cadena de Frío en los Almacenes de una Empresa Logística de Alimentos", Tesis de Grado, Univ. Catol. Andres Bello, Caracas, 2010. Disponible en: http://biblioteca2.ucab.edu.ve/anexos/biblioteca/marc/texto/AAS1535.pdf
[10] Food and Agriculture Organization of the United Nations. "Programa conjunto fao/oms sobre normas alimentarias comité del codex sobre higiene de los alimentos". Home | Food and Agriculture Organization of the United Nations. [En línea]. Disponible: https://www.fao.org/fao-who-codexalimentarius/sh-proxy/fr/?lnk=1&url=https%3A%2F%2Fworkspace.fao.org%2Fsites%2Fcodex%2FMeetings%2FCX-712-54%2FCRDs%2Ffh54_crd06s.pdf
[11] Food and Agriculture Organization of the United Nations. "Código internacional recomendado revisado de practicas-principios generales de higiene de los alimentos". Home | Food and Agriculture Organization of the United Nations. [En línea]. Disponible: https://www.fao.org/4/w6419S/w6419s0o.htm
[12] Food and Agriculture Organization of the United Nations. "Tabla de almacenamiento de alimentos en frío". Home | Food and Agriculture Organization of the United Nations. [En línea]. Disponible: https://www.fao.org/fileadmin/user_upload/rlc/come-sano/t_es.pdf
[13] Food and Agriculture Organization of the United Nations y A. F. López Camelo, Manual para la preparación y venta de frutas y hortalizas. del campo al mercado - Capítulo 3. Almacenamiento. Food & Agriculture Org. United Nations, 2003. [En línea]. Disponible: https://www.fao.org/4/y4893s/y4893s00.htm#Contents
[14] Jones, C. B. "Systematic software development using VDM". Vol. 2. Englewood Cliffs: Prentice Hall, 1990.. Disponible: https://www.di.uminho.pt/~jno/ps/Jones1990.pdf
[15] P. Gorm Larssen."Formal Methods Europe". Accedido el 22 de noviembre de 2024. [En línea]. Disponible: https://fmeurope.org/conferences/fm2006/VDM++%20tutorial%20FM%202006%20handouts.pdf
[16] J. Fitzgerald, P. Gorm Larsen, P. Mukherjee, N. Plat, and M. Verhoef, "Validated Designs for Object Oriented Systems". Springer, New York, 2005.
[17] CSK SYSTEMS, "The VDM++ Language". Technical Report, 2009
[18] A. Müller, "VDM — The Vienna Development Method," Bachelor's thesis in "Formal Methods in Software Engineering," Johannes Kepler University Linz, Apr. 20, 2009.
[19] E. Vidal-Duarte, "Aplicación y validación de especificaciones formales ligeras en el modelo conceptual: Reduciendo la ambigüedad e incrementando la conformidad entre los requerimientos y el código", Infonor Chile 2012, p. 9, 2012. [En línea]. Disponible: https://infonorchile2012.uta.cl/infonor2012_155187.pdf
[20] CSK SYSTEMS d. VDM++ Toolbox User Manual. Technical Report, 2009
[21] Endress+Hauser. "Medición de temperatura RTD". Caudal, nivel, análisis de líquidos, análisis óptico, presión, medición de temperatura, software y productos de sistema | Endress+Hauser. [En línea]. Disponible: https://www.es.endress.com/es/instrumentacion-campo/tecnologias-de-medicion/medición-temperatura-rtd